

LEGADO

Blueprint de sistema, modelo de datos y APIs

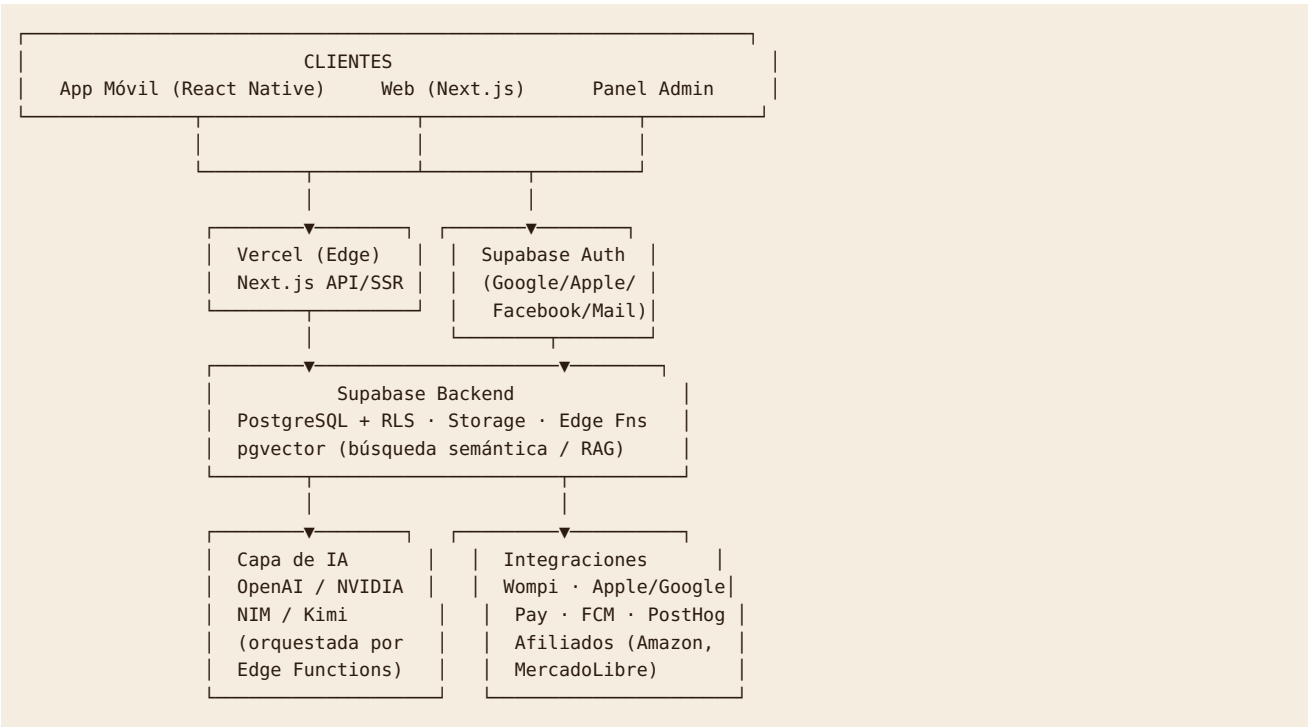
Roberto — Fundador

Julio 2026

- [1. Visión de arquitectura](#)
- [2. Modelo de datos \(entidades principales\)](#)
 - [2.1 Núcleo de contenido](#)
 - [2.2 Geografía y cultura](#)
 - [2.3 Usuarios y comunidad](#)
 - [2.4 IA y personalización](#)
 - [2.5 Comercial](#)
- [3. Seguridad y roles \(RLS\)](#)
- [4. API \(resumen de endpoints clave — Fase 1\)](#)
- [5. Infraestructura y entornos](#)
- [6. Consideraciones de costo de IA](#)
- [7. Roadmap técnico \(alineado con Roadmap Backlog.pdf\)](#)

1. Visión de arquitectura

Arquitectura **serverless, mobile-first, IA-nativa**, optimizada para un equipo pequeño (1 fundador + agentes IA) que necesita velocidad de iteración sin sacrificar escalabilidad futura.



Principios de diseño: 1. **Un solo backend (Supabase)** para auth, datos, storage y funciones — minimiza superficie operativa para un equipo pequeño. 2. **IA orquestada en el backend**, nunca con llaves de API expuestas en el cliente. 3. **RLS (Row Level Security)** en cada tabla desde el día uno — el modelo de roles (sección 17 del PRD) se aplica a nivel de base de datos, no solo de UI. 4. **Todo cacheable:** contenido editorial (recetas, historias) se sirve como contenido mayormente estático/cacheado; solo lo personalizado (recomendaciones, listas de compra, progreso) golpea la base de datos en tiempo real. 5. **Costo de IA controlado:** generación (imágenes, voz, texto) se cachea y se asocia a la entidad (receta, usuario) para no regenerar contenido repetido.

2. Modelo de datos (entidades principales)

2.1 Núcleo de contenido

recipes | Campo | Tipo | Notas | |—|—|—| | id | uuid | PK | | title | text | | | title_ar | text | nullable — nombre en árabe/transliteración | | slug | text | único, para SEO | | origin_city_id | uuid | FK → **cities** | | history_text | text | narrado por Abuela Tula / Tío Roberto | | arab_influence_text | text | explica técnica/ingrediente de origen árabe | | difficulty | enum | fácil / media / difícil | | prep_time_minutes | int | | | cost_estimate | enum | bajo / medio / alto | | servings_base | int | usado para escalar porciones | | status | enum | borrador / en_revisión / publicada / archivada | | created_by | uuid | FK → **users** | | approved_by | uuid | FK → **users**, nullable | | is_premium | bool | | | created_at / updated_at | timestamptz | |

recipe_ingredients — id, recipe_id, ingredient_id, cantidad, unidad, es_opcional, nota_sustitución
ingredients — id, nombre, nombre_ar, historia_breve, alérgenos (array), embedding (pgvector, para búsqueda semántica) **recipe_steps** — id, recipe_id, orden, texto, timer_seconds (nullable), audio_url (nullable) **recipe_media** — id, recipe_id, tipo (foto/video/audio), url, generado_por_ia (bool), fuente

2.2 Geografía y cultura

cities — id, nombre, lat, lng, descripción, región (Caribe/Guajira/etc.) **routes** (rutas gastronómicas) — id, city_id, nombre, descripción, recetas asociadas (m:n vía **route_recipes**) **stories** (historias/artículos editoriales) — id, título, cuerpo, tipo (historia_ingredientes / inmigración / técnica), recetas relacionadas, fuentes_bibliográficas (jsonb), timeline_year (nullable) **timeline_events** — id, año, título, descripción, story_id (nullable)

2.3 Usuarios y comunidad

users — id, auth_provider, nombre, email, rol (admin/editor/chef/historiador/nutricionista/fotógrafo/moderador/usuario), plan (gratis/premium), país, created_at **favorites** — user_id, recipe_id **user_recipes** (contribuciones) — igual estructura que **recipes** + user_id, estado de aprobación **ratings** — user_id, recipe_id, estrellas, comentario **follows** — follower_id, followed_id (chefs/usuarios) **badges** / **user_badges** — sistema de insignias y niveles **challenges** / **user_challenges** — retos (ej. “30 días cocinando costeño”)

2.4 IA y personalización

ai_generations — id, tipo (sustitución/nutrición/imagen/audio/libro), entidad_relacionada, prompt, resultado, costo_estimado, created_at (*auditoría y control de costo*) **shopping_lists** — id, user_id, items (jsonb), origen (receta/menú semanal) **weekly_menus** — id, user_id, recetas (array), semana **user_embeddings** — user_id, embedding (pgvector) — para recomendaciones basadas en preferencias

2.5 Comercial

subscriptions — user_id, plan, proveedor_pago, estado, renueva_en **courses** / **course_enrollments** / **certificates** **marketplace_links** (afiliados) — id, producto, proveedor (Amazon/MercadoLibre/local), url, categoría **events** — id, nombre (Festival del Frito, Ramadán, etc.), fecha, descripción, tipo (digital/presencial)

3. Seguridad y roles (RLS)

- Cada tabla sensible tiene políticas RLS por rol: por ejemplo, `recipes` en estado borrador solo visible para `editor`, `chef`, `historiador` y `admin` asignados; `publicada` visible para todos.
- Contenido premium (`is_premium = true`) filtrado a nivel de política según `subscriptions.estado = activo`.
- Auditoría: toda acción de aprobación/publicación queda registrada (`approved_by`, `timestamps`) — trazabilidad para el rol Historiador/Editor.
- Llaves de proveedores de IA y pagos **solo viven en Edge Functions**, nunca en el cliente móvil/web.

4. API (resumen de endpoints clave — Fase 1)

Todas expuestas como Supabase Edge Functions / RPC, consumidas por el propio cliente (API interna, ver PRD sección 19):

- `GET /recipes` — listado con filtros (ciudad, dificultad, premium, búsqueda semántica)
- `GET /recipes/:slug` — ficha completa
- `POST /ai/substitute-ingredient` — sustitución IA
- `POST /ai/scale-recipe` — conversión de porciones
- `POST /ai/shopping-list` — generar lista de compras
- `POST /ai/weekly-menu` — generar menú semanal
- `POST /ai/recommend` — recomendaciones personalizadas (RAG sobre `user_embeddings`)
- `POST /ai/generate-legacy-book` — Libro LEGADO (PDF) — Premium
- `POST /recipes/:id/rate` — calificación
- `POST /user-recipes` — contribución de receta de comunidad
- `GET /routes/:city` — ruta gastronómica por ciudad

5. Infraestructura y entornos

- **Entornos:** `dev` → `staging` → `production`, cada uno con su propio proyecto Supabase.
- **CI/CD:** GitHub Actions → build/test → deploy automático a Vercel (web) y build de React Native vía EAS (Expo Application Services) para móvil.
- **Observabilidad:** PostHog (producto), Supabase Logs + Sentry (errores).
- **Backups:** snapshots diarios automáticos de PostgreSQL (Supabase).

6. Consideraciones de costo de IA

Feature	Estrategia de costo
Generación de imágenes	Bajo demanda + caché permanente por receta (se genera una vez, se reutiliza)
Voz/narración	Pre-generada para catálogo fundacional; bajo demanda para contenido de usuarios
Chef IA conversacional	Modelo económico (Kimi/NIM) para consultas simples; modelo más potente (OpenAI) solo para casos complejos — enrutamiento por Edge Function
Reconocimiento de plato por foto	Rate-limited por usuario/día en plan gratis; ilimitado en Premium

7. Roadmap técnico (alineado con Roadmap_Backlog.pdf)

1. **Fase 0:** esquema de base de datos, RLS, panel admin (CMS), carga de 30-50 recetas.
2. **Fase 1:** apps móvil/web, auth social, Modo Cocina, IA core, pagos (Wompi/Apple/Google Pay).
3. **Fase 2:** contribuciones de usuarios, gamificación, chef IA conversacional, reconocimiento de plato, cursos.
4. **Fase 3:** certificados, marketplace, API pública B2B, árbol genealógico de recetas.